

Deep Learning for teens

By Hamid Sadeghi

Table of contents

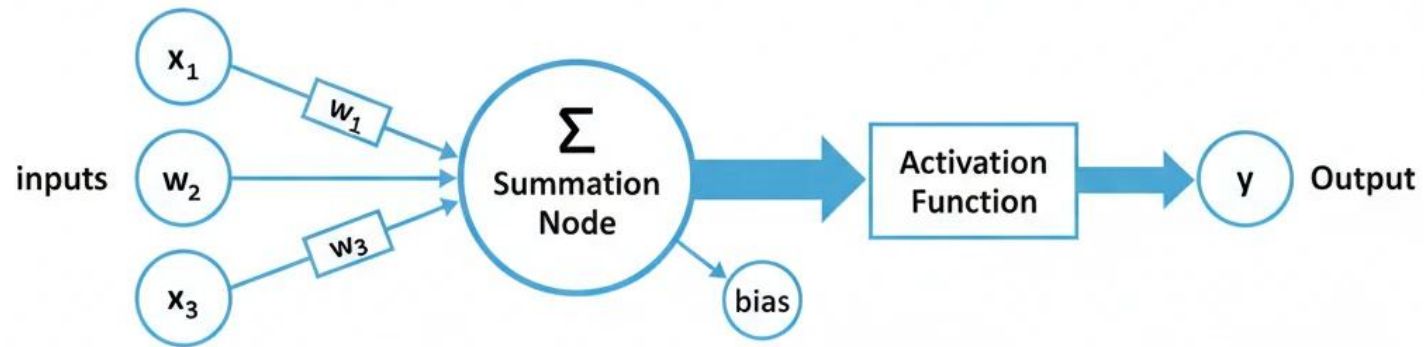
- ▶ make_moons dataset
- ▶ What is a neuron
- ▶ Layers in NN
- ▶ Neural network explanation
- ▶ Activation function
- ▶ Loss
- ▶ Forward Pass
- ▶ Backward error propagation
- ▶ Gradient descent
- ▶ decision boundary
- ▶ Homework

make_moons dataset

- ▶ The make_moons dataset is a simple, synthetic dataset in scikit-learn that generates two interleaving half-circles (often called “moons”).

What is a neuron

- Smallest block of a neural network



Layers in NN

- ▶ Every **neural network (NN)** can have a **different number of layers**
 - ▶ First layer
 - ▶ input layer
 - ▶ last layer
 - ▶ output layer
 - ▶ The layers in between
 - ▶ hidden layers

Neural network explanation

- ▶ We split data into three parts
 - ▶ Training data
 - ▶ Used to teach the model
 - ▶ Validating data
 - ▶ Used during training to tune the model
 - ▶ not to update weights, but to monitor performance and guide decisions like
 - ▶ Test data
 - ▶ Used after training to check how well the model performs
 - ▶ it's never seen by the model during training, so it gives a **true measure of generalization**

Activation function

- ▶ An **activation function** decides whether a neuron should “fire” or not

Loss

Measure how far predictions are from the true labels.

Example (binary cross-entropy):

$$L = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Forward Pass

- ▶ Forward pass: input goes through the network to get output.
- ▶ take input → pass it through the layers → get output.

Backward error propagation

- ▶ **Backward error propagation**, or **backpropagation**, is the algorithm that computes how much each weight in a neural network contributed to the prediction error, so we can adjust those weights to reduce future errors.
- ▶ Steps:
 - ▶ Calculating the loss gradient at the output layer
 - ▶ Propagating that error backward through each layer using the chain rule
 - ▶ Updating weights using gradient descent

Graduate descent

- ▶ Gradient descent is an optimization algorithm that updates weights by moving in the direction that reduces loss.
- ▶ Initialize Parameters
 - ▶ Start with random weights W and biases b
- ▶ Forward Pass
 - ▶ Compute predictions by passing inputs through the network
- ▶ Compute Loss
 - ▶ Measure how far predictions are from the true labels.
- ▶ Backward Pass (Gradients)
- ▶ Update Parameters
 - ▶ Adjust weights and biases using the gradient:

$$z = X \cdot W + b$$

$$a = f(z) \quad (\text{activation like ReLU or sigmoid})$$

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial L}{\partial W}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \cdot \frac{\partial L}{\partial b}$$

decision boundary

- ▶ Imagine you have two groups of points: **red dots** and **blue dots**
- ▶ A decision boundary is the line (or curve) the model draws to separate the red from the blue.

Homework

- ▶ I want you to use the Iris dataset to build a neural network using TensorFlow.
- ▶ Split the data, and calculate accuracy for both the training and testing sets.
- ▶ Check for overfitting. Plot the loss and accuracy for the training and validation sets.
- ▶ Try different numbers of epochs to see how training duration affects performance.
- ▶ Finally, plot the decision boundary.